



มหาวิทยาลัยขอนแก่น

วิทยา จริยา ปัญญา

KHON KAEN UNIVERSITY



Python and MySQL Database

Assoc. Prof. Dr. Kanda Runapongsa Saikaew

Faculty of Engineering

Khon Kaen University



คณะวิศวกรรมศาสตร์ มหาวิทยาลัยขอนแก่น

FACULTY OF ENGINEERING KHON KAEN UNIVERSITY

Agenda

- Comparing MySQL to Other SQL Databases
- Installing MySQL Connector/Python
- Establishing a Connection with MySQL Server
- Creating a Table
- Inserting Records in Tables
- Reading Records from the Database
- Updating and Deleting Records from the Database



Prerequisites and Setup

Before we begin:

- MySQL Server 8.0+ installed and running
- Python 3.7+ installed
- Basic SQL knowledge (SELECT, INSERT, UPDATE, DELETE)
- Understanding of relational database concepts

Installation checklist:

1. Install MySQL Server
2. Create a test database and user account
3. Install Python MySQL connector:
pip install mysql-connector-python
4. Test MySQL connection using command line client

Verify your setup:

mysql --version

python --version

```
~/courses/en813304/exercises/kanda-en813304-db-exercises$mysql --version
mysql  Ver 9.7.1 for macos26.4 on arm64 (Homebrew)
~/courses/en813304/exercises/kanda-en813304-db-exercises$python3 --version
Python 3.9.6
~/courses/en813304/exercises/kanda-en813304-db-exercises$
```



Comparing MySQL to Other SQL Databases

- MySQL is one of the most popular database management systems (DBMSs) on the market today
- It ranked second only to the Oracle DBMS in this year's [DB-Engines Ranking](#)

Rank			DBMS	Database Model
Sep 2026	Aug 2026	Sep 2025		
1.	1.	1.	Oracle	Relational, Multi-model ⓘ
2.	2.	2.	MySQL	Relational, Multi-model ⓘ
			Microsoft SQL Server	Relational, Multi-model ⓘ
3.	3.	3.	PostgreSQL	Relational, Multi-model ⓘ
4.	4.	4.	MongoDB	Document, Multi-model ⓘ
5.	5.	5.	Snowflake	Relational
6.	6.	6.	Databricks	Multi-model ⓘ
7.	7.	↑ 9.	Redis	Key-value, Multi-model ⓘ
8.	8.	↓ 7.	IBM Db2	Relational, Multi-model ⓘ
9.	9.	↓ 8.	SQLite	Relational
10.	↑ 11.	↑ 11.		



Installing MySQL Connector/Python (1/2)

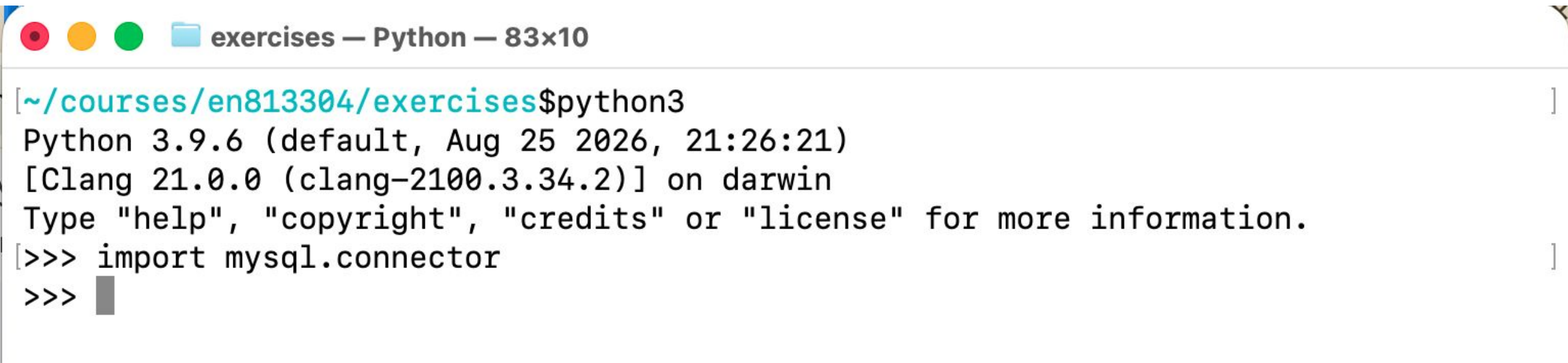
- A database driver is a piece of software that allows an application to connect and interact with a database system
- Programming languages like Python need a special driver before they can speak to a database from a specific vendor
- Use the command **pip install mysql-connector-python**

```
[~/courses/en813304/exercises/mysql]$ pip3 install mysql-connector-python
Defaulting to user installation because normal site-packages is not writeable
Collecting mysql-connector-python
  Downloading mysql_connector_python-9.0.0-cp39-cp39-macosx_13_0_arm64.whl.metadata (2.0 kB)
  Downloading mysql_connector_python-9.0.0-cp39-cp39-macosx_13_0_arm64.whl (13.4 MB)
  ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 13.4/13.4 MB 521.6 kB/s eta 0:00:00
Installing collected packages: mysql-connector-python
Successfully installed mysql-connector-python-9.0.0
```



Installing MySQL Connector/Python (2/2)

- To test if the installation was successful, type **import mysql.connector** on your Python terminal

A screenshot of a macOS terminal window titled "exercises — Python — 83x10". The terminal shows the execution of a Python script. The prompt is a blue path: [~/courses/en813304/exercises\$. The command python3 is entered, followed by the Python 3.9.6 startup banner: Python 3.9.6 (default, Aug 25 2026, 21:26:21) [Clang 21.0.0 (clang-2100.3.34.2)] on darwin. Then, the prompt Type "help", "copyright", "credits" or "license" for more information. is shown. Finally, the Python prompt [>>> is shown, followed by the command import mysql.connector, and another [>>> prompt with a cursor, indicating the command was executed successfully without any error messages.

```
exercises — Python — 83x10  
[~/courses/en813304/exercises$python3  
Python 3.9.6 (default, Aug 25 2026, 21:26:21)  
[Clang 21.0.0 (clang-2100.3.34.2)] on darwin  
Type "help", "copyright", "credits" or "license" for more information.  
[>>> import mysql.connector  
>>>
```



Establishing a Connection with MySQL Server

1. Connect to the MySQL server
2. Create a new database
3. Connect to the newly created or an existing database
4. Execute a SQL query and fetch results
5. Inform the database if any changes are made to a table
6. Close the connection to the MySQL server

Sample codes can be downloaded from [this link](#)



Establishing a Connection

```
establish.py M X
python > establish.py > ...
You, 5 minutes ago | 1 author (You)
1 from getpass import getpass
2 from mysql.connector import connect, Error
3
4 # Initialize connection variable for proper cleanup
5 connection = None
6
7 try:
8     # Establish database connection
9     # Never hardcode credentials - always prompt user
10    connection = connect(
11        host="localhost",
12        user=input("Enter username: "),
13        password=getpass("Enter password: "), # Hidden password input
14        database=input("Enter database name: "),
15    )
16
17 # Verify successful connection
18 if connection.is_connected():
19     print("Database connection successful!")
20     # Your database operations would go here
21
22 except Error as e:
23     # Handle any connection errors that might occur
24     print(f"Connection error: {e}")
25
26 finally:
27     # Always close connection to prevent resource leaks
28     if connection and connection.is_connected():
29         connection.close()
30     print("Database connection closed.")
31
```

OUTPUT TERMINAL PORTS GITLENS DEBUG CONSOLE

```
~/courses/en813304/exercises/kanda-en813304-db-exercises$clear
~/courses/en813304/exercises/kanda-en813304-db-exercises$/usr/bin/python3 /Users/kandasaikaew/courses/en813304/exercises/python/establish.py
Enter username: root
Enter password:
Enter database name: student_5829_db
Database connection successful!
Database connection closed.
```

```
establish.py M X
python > establish.py > ...
16
17 # Verify successful connection
18 if connection.is_connected():
19     print("Database connection successful!")
20     # Your database operations would go here
21
22 except Error as e:
23     # Handle any connection errors that might occur
24     print(f"Connection error: {e}")
25
26 finally:
27     # Always close connection to prevent resource leaks
28     if connection and connection.is_connected():
29         connection.close()
30     print("Database connection closed.")
31
```

OUTPUT TERMINAL PORTS GITLENS DEBUG CONSOLE

```
~/courses/en813304/exercises/kanda-en813304-db-exercises$clear
~/courses/en813304/exercises/kanda-en813304-db-exercises$/usr/bin/python3 /Users/kandasaikaew/courses/en813304/exercises/python/establish.py
Enter username: root
Enter password:
Enter database name: student_5829_db
Database connection successful!
Database connection closed.
```



Explaining the Code in Establishing the Connection

- You should always deal with the exceptions that might be raised while establishing a connection to the MySQL server
 - Use a try...except block to catch and print any exceptions
- You should always close the connection after you're done accessing the database
 - Leaving unused open connections can lead to errors and performance issues
- You should never hard-code your login credentials
 - Prompts the user for login credentials or uses environment variables



Creating a New Database

- To execute a SQL query in Python, you'll need to use a cursor, which abstracts away the access to database records
- MySQL Connector/Python provides you with the [MySQLCursor](#) class, which instantiates objects that can execute MySQL queries in Python
- An instance of the MySQLCursor class also called a [cursor](#)
- cursor objects make use of a MySQLConnection object to interact with your MySQL server



Example in Creating a New Database

```
establish.py M crt_db.py M X
python > crt_db.py > ...
You, 13 seconds ago | 1 author (You)
1 # Import modules for secure input and MySQL connection
2 from getpass import getpass
3 from mysql.connector import connect, Error
4
5 # Initialize connection variable
6 connection = None
7
8 try:
9     # Connect to MySQL server with user credentials
10    connection = connect(
11        host="localhost",
12        user=input("Enter username: "),
13        password=getpass("Enter password: "), # Hidden password input
14    )

```

OUTPUT TERMINAL PORTS GITLENS DEBUG CONSOLE

```
~/courses/en813304/exercises/kanda-en813304-db-exercises$ /usr/bin/python3 /Users/kandasaikaew/courses/en813304/exercises/
anda-en813304-db-exercises/python/crt_db.py
Enter username: root
Enter password:
Database created successfully
MySQL connection closed
```

```
establish.py M crt_db.py M
python > crt_db.py > ...
16 # SQL query to create database if it doesn't exist
17 create_db_query = "CREATE DATABASE IF NOT EXISTS movies_db"
18
19 # Execute the database creation query You, 4 minutes ago
20 cursor = connection.cursor()
21 cursor.execute(create_db_query)
22 print("Database created successfully")
23
24 # Close cursor
25 cursor.close()
26
27 # Handle MySQL connection and execution errors
28 except Error as e:
29     print(f"Error: {e}")
30
31 finally:
32     # Ensure connection is closed even if error occurs
33     if connection and connection.is_connected():
34         connection.close()
35         print("MySQL connection closed")
36

```



Database Names and Python Filenames

- **Database Names**

- **Recommended names:** Start with a letter, use lowercase `snake_case`, and put any numbers at the end (for example `school_2026`, not `2026_school`).
- **Why not start with a number:** Standard SQL and most DBMSs (for example PostgreSQL) require identifiers to start with a letter
 - MySQL allows a leading digit, but names like `1e5` get parsed as numbers.

- **Python Filenames**

- **Recommended names:** Follow PEP 8: short, all-lowercase module names, with underscores if that helps readability (for example `db_utils.py`, `lab_2026.py`)
 - Avoid hyphens, spaces, and names that clash with standard modules (for example `random.py`, `mysql.py`).
- **Why not start with a number:** A module name must be a valid Python identifier to be imported, and identifiers can't start with a digit. So `import 2026_lab` is a `SyntaxError`, and the file can only be run directly or loaded awkwardly via `importlib.import_module("2026_lab")`



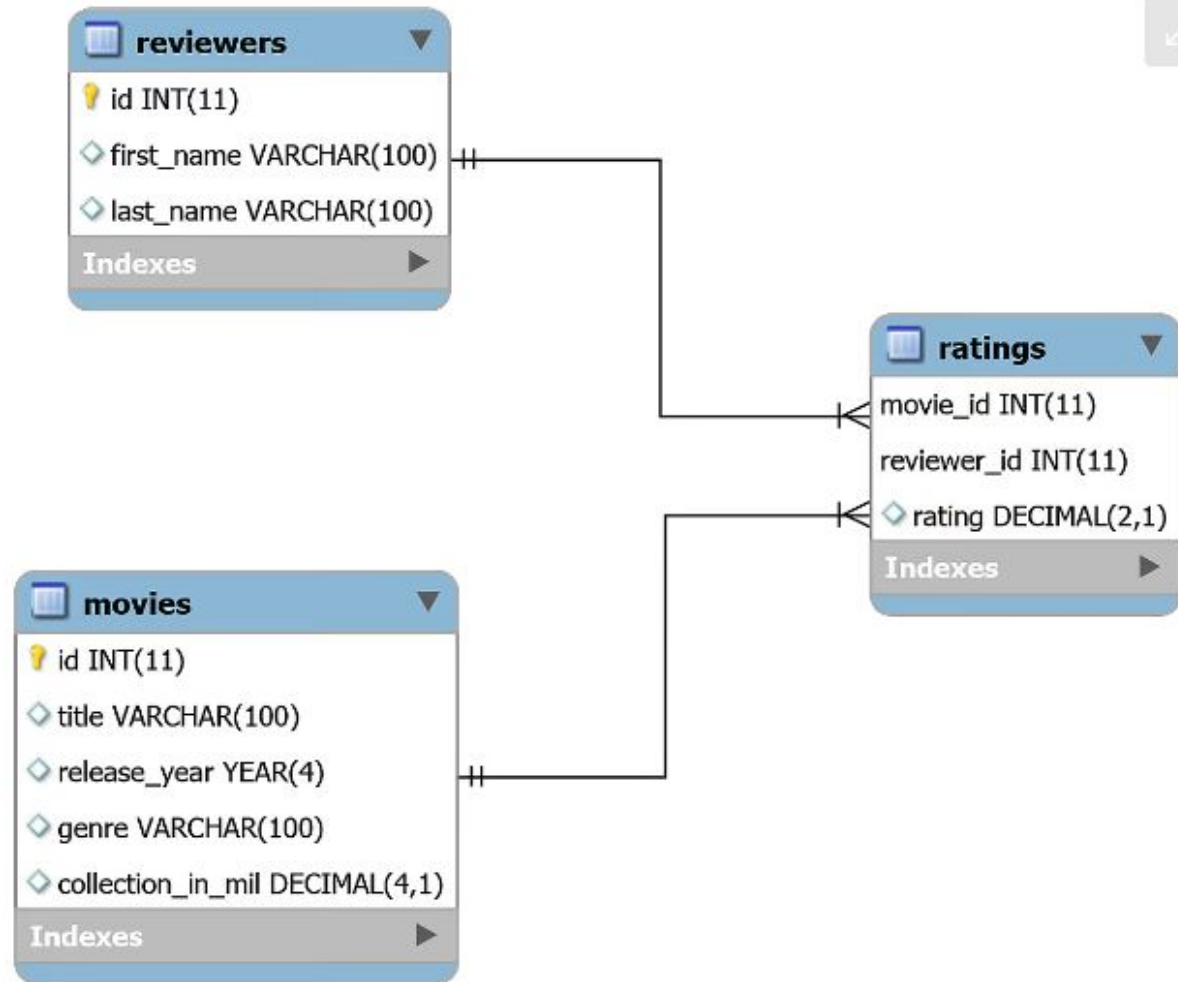
Database Cursor Concepts

- **Cursor as Database Pointer**
 - A cursor acts as a database pointer that executes SQL commands and manages result sets
 - Think of it as your direct interface to send commands to the database
 - Created from an existing database connection
- **Execute vs Fetch Pattern**
 - Always follow the two-step process: `cursor.execute()` sends the query to the database server
 - Then `cursor.fetchall()`, `fetchone()`, or `fetchmany()` retrieves results into Python memory



Connecting to an Existing Database

- You can do this using the same connect() function that you used earlier by sending an additional parameter called database



Creating Tables Using the CREATE TABLE Statement

- To create a new table, you need to pass this query to `cursor.execute()`, which accepts a MySQL query and executes the query on the connected MySQL database

```
# Create movies table with auto-increment primary key
create_movies_table_query = """
    CREATE TABLE IF NOT EXISTS movies(
        id INT AUTO_INCREMENT PRIMARY KEY,
        title VARCHAR(100),
        release_year YEAR(4),
        genre VARCHAR(100),
        collection_in_mil INT
    )
    """

with connection.cursor() as cursor:
    cursor.execute(create_movies_table_query)
    connection.commit() # Save changes to database
print("Table 'movies' created successfully")
```

- You pass `create_movies_table_query` to `cursor.execute()`, which performs the required execution
- Notice the `connection.commit()` saves changes to database
- By default, **your MySQL connector doesn't autocommit transactions**



Creating Table Reviewers

```
30 | # Create reviewers table with personal info
31 | create_reviewers_table_query = """
32 |     CREATE TABLE IF NOT EXISTS reviewers(
33 |         id INT AUTO_INCREMENT PRIMARY KEY,
34 |         first_name VARCHAR(100),
35 |         last_name VARCHAR(100)
36 |     )
37 |     """
38 |
39 | with connection.cursor() as cursor:
40 |     cursor.execute(create_reviewers_table_query)
41 |     connection.commit() # Save changes to database
42 | print("Table 'reviewers' created successfully")
```



Creating Table Ratings

```
44 | # Create ratings table with foreign key relationships
45 | create_ratings_table_query = """
46 |     CREATE TABLE IF NOT EXISTS ratings(
47 |         movie_id INT,
48 |         reviewer_id INT,
49 |         rating DECIMAL(2,1),
50 |         FOREIGN KEY(movie_id) REFERENCES movies(id),
51 |         FOREIGN KEY(reviewer_id) REFERENCES reviewers(id),
52 |         PRIMARY KEY(movie_id, reviewer_id)
53 |     )
54 | """
55 |
56 | with connection.cursor() as cursor:
57 |     cursor.execute(create_ratings_table_query)
58 |     connection.commit() # Save changes to database
59 | print("Table 'ratings' created successfully")
60 |
61 | print("All tables created successfully!")
```

```
63 | # Handle MySQL connection and execution errors
64 | except Error as e:
65 |     print(f"Error: {e}")
66 |     # Connection is still automatically closed even if error occurs
67 |
68 | finally:
69 |     # Ensure connection is closed even if error occurs
70 |     if connection and connection.is_connected():
71 |         connection.close()
72 |     print("MySQL connection closed")
```

OUTPUT TERMINAL PORTS GITLENS DEBUG CONSOLE

NameError: name 'connection' is not defined

```
• ~/courses/en813304/exercises/kanda-en813304-db-exercises$/u
anda-en813304-db-exercises/python/crt_tables.py
Enter username: root
Enter password:
Connected successfully to movies_db
Table 'movies' created successfully
Table 'reviewers' created successfully
Table 'ratings' created successfully
```



Context Manager

Context Manager (**with** statement)

- Automatically closes the cursor when done (even if error occurs)
- Best practice for resource management
- Prevents memory leaks and connection issues
- What 'with' statement cleans up:
 - **Cursor.close()** → Frees result buffers and cursor resources
 - **Connection.close()** → Terminates database connection



Describing a Table

```
describe_table.py M x
python > describe_table.py > ...

4 try:
5     with connect(
6         host="localhost",
7         user=input("Enter username: "),
8         password=getpass("Enter password: "),
9         database=input("Enter database name: "),
10    ) as connection:
11        print(f"Connect successfully at {connection}")
12        show_movies_table_query = "DESCRIBE courses"
13        with connection.cursor() as cursor:
14            cursor.execute(show_movies_table_query)
15            description = cursor.fetchall()
16            for row in description:
17                print(row)
18
```

OUTPUT TERMINAL PORTS GITLENS DEBUG CONSOLE

```
• ~/courses/en813304/exercises/kanda-en813304-db-exercises$ /usr/bin/python3 /Users/kandasai/...
anda-en813304-db-exercises/python/describe_table.py
Enter username: root
Enter password:
Enter database name: student_5829_db
Connect successfully at <mysql.connector.connection_cext.CMySQLConnection object at 0x104dac...
('course_id', 'varchar(8)', 'NO', 'PRI', None, '')
('course_name', 'varchar(100)', 'NO', '', None, '')
('credits', 'int', 'YES', '', None, '')
('department', 'varchar(50)', 'YES', '', None, '')
```

- Now, that you've created all three tables, you can look at their schema using the following SQL statement: `DESCRIBE <table_name>;`
- To get some results back from the cursor object, you need to use `cursor.fetchall()`

Execute vs Fetch Pattern:

1. `execute()` - Sends query to database, results stay on server
2. `fetchall()` - Retrieves all results from server to Python memory
3. Always fetch AFTER execute, never before



fetchall() and Alternative Approaches

- `fetchall()` returns a **list of tuples**
- Each tuple represents one database row
- Access columns by index: `row[0]`, `row[1]`, etc.

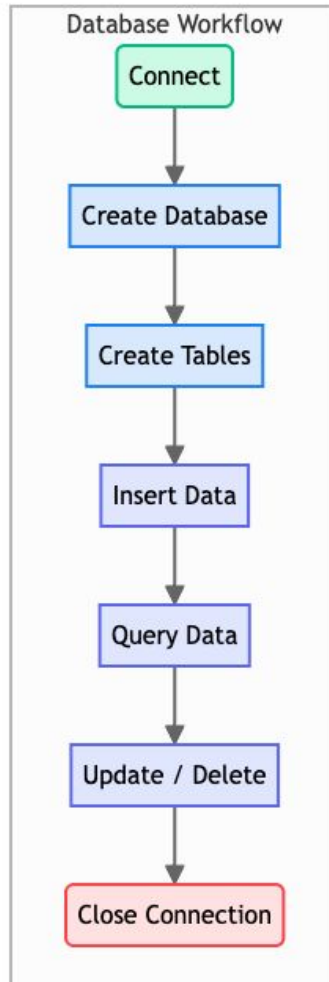
Alternative Approaches:

- `fetchone()` - Get one row at a time (good for large datasets)
- `fetchmany(n)` - Get specific number of rows



Database Workflow

Workflow Steps Explained



1. **Connect:** Establish the initial connection to the database server.
2. **Create Database:** Create a new database container for the data.
3. **Create Tables:** Define the structure of the data by creating tables with specific columns.
4. **Insert Data:** Populate the tables with initial records.
5. **Query Data:** Retrieve and read data from the tables using commands like `SELECT`.
6. **Update / Delete:** Modify or remove existing records in the tables as needed.
7. **Close Connection:** Terminate the connection to free up server resources.



Database Cursors Explains

- **Cursor definition:** A database pointer that executes SQL commands and manages result sets
- **Connection vs Cursor:** Connection manages the database session; cursor executes specific queries
- **Execute-fetch pattern:** Always execute() first to send query, then fetch() to retrieve results
- **Cursor lifecycle:** Create from connection → execute query → fetch results → close cursor
- **Multiple cursors:** One connection can create multiple cursors for different operations



Credential Management Approaches

- **Hard-coded credentials:** Simple but insecure - never use in production environments
- **Interactive prompts:** Secure but inconvenient - good for development and one-time scripts
- **Environment variables:** Secure and flexible - industry standard for production deployments
- **Configuration files (.env):** Developer-friendly with proper .gitignore setup - ideal for team development
- **Enterprise solutions:** Key management systems (AWS Secrets Manager, Azure Key Vault) for large-scale applications



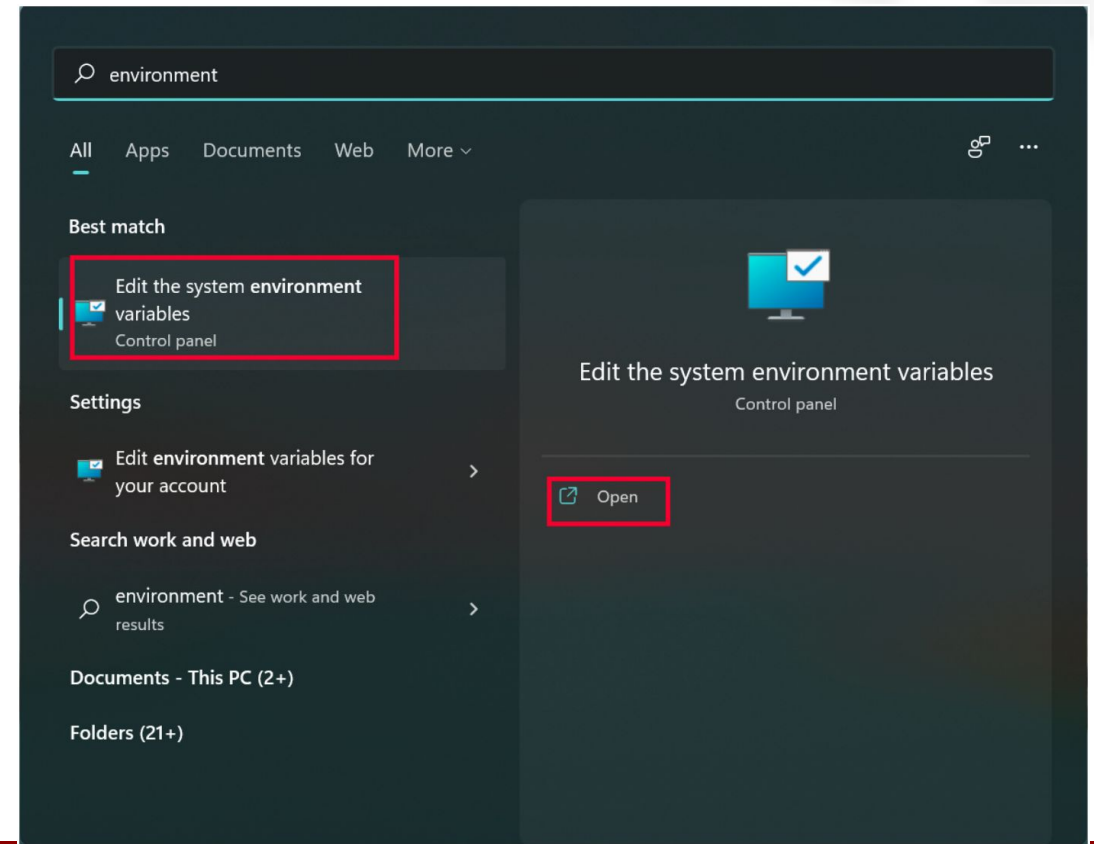
Connection Management Best Practices

- **Environment Variables:**
 - Store database credentials in environment variables or `.env` files, never hardcode passwords in source code for security and portability
- **Transaction Control:**
 - Remember that MySQL connector doesn't autocommit by default - always call `connection.commit()` after INSERT, UPDATE, or DELETE operations to save changes
- **Error Handling:**
 - Wrap database operations in try-except blocks to handle connection failures, SQL errors, and unexpected exceptions gracefully
- **Connection Cleanup:**
 - Use `finally` blocks or context managers to ensure connections are closed even when errors occur, preventing database connection pool exhaustion



Secure Credential Management: Setting Environment Variables (System Level)

- Before running your Python script, you need to set the environment variables.
- This can be done in different ways depending on your operating system
- For Unix-like systems (Linux, macOS), edit `.zshrc` in home directory
- # In `.zshrc` or `.bashrc` - USE PLACEHOLDERS ONLY
- `export DB_USERNAME="your_username_here"`
`export DB_PASSWORD="your_password_here"`
- For Windows system, set environment variables



Secure Credential Management: .env File (Recommended for projects) (1/2)

1. Install: **pip install python-dotenv**
2. Create .env file with actual credentials
File: .env (no extension)
DB_HOST=localhost
DB_USERNAME=root
DB_PASSWORD=your_actual_password
DB_DATABASE=movies_db
3. Load in Python: from dotenv
import load_dotenv load_dotenv()

```
python > connect_via_env.py > ...
1  import os
2  from dotenv import load_dotenv
3  from mysql.connector import connect, Error
4
5  # Load environment variables from .env file
6  env_path = os.path.join(os.path.dirname(__file__), ".env")
7  print(f"env_path is {env_path}")
8  load_dotenv(dotenv_path=env_path, override=True)
9
10 try:
11     with connect(
12         host=os.getenv("DB_HOST"),
13         user=os.getenv("DB_USERNAME"),
14         password=os.getenv("DB_PASSWORD"),
15         database=os.getenv("DB_DATABASE"),
16     ) as connection:
17         print(f"Connected successfully at {connection}")
18
19 except Error as e:
20     print(f"Error: {e}")
21
```



Secure Credential Management: .env File (Recommended for projects) (2/2)

4. Add .env to .gitignore

```
[~/courses/en813304/exercises/kanda-en813304-db-exercises]$ more .gitignore
# File: .gitignore
.env
__pycache__/*
*.pyc
```

5. Create .env.example with placeholders for team sharing

Correct format

DB_HOST=localhost

DB_USERNAME=root

DB_PASSWORD=mypassword123

DB_DATABASE=movies_db

Incorrect formats that cause issues:

DB_PASSWORD = mypassword123 (spaces around =)

DB_PASSWORD="mypassword123" (quotes not needed)



Data Insertion Strategies

- **Single record insertion:** Use `.execute()` for one-off records or when data varies significantly
- **Batch insertion:** Use `.executemany()` for multiple similar records - much more efficient
- **Performance difference:** `.executemany()` can be 10-100x faster than multiple `.execute()` calls
- **Memory considerations:** Large datasets may require chunking to avoid memory overflow
- **Transaction safety:** Both methods require `connection.commit()` to permanently save changes



Inserting Records in Tables

- The first method, `.execute()`, works well when the number of records is small and the records can be hard-coded
 - The first approach uses the same `cursor.execute()` method that you've been using until now
 - You write the `INSERT INTO query` in a string and pass it to `cursor.execute()`
 - You can use this method to insert data into the movies table.
- The second method, `.executemany()`, is more popular and is better suited for real-world scenarios



Inserting Data Using Method `.executemany()`

- The previous approach is more suitable when the number of records is fairly small and you can write these records directly into the code
- But this is rarely true. You'll often have this data stored in some other file, or the data will be generated by a different script and will need to be added to the MySQL database
- This is where `.executemany()` comes in handy
- It accepts two parameters:
 1. A **query** that contains placeholders for the records that need to be inserted
 2. A **list** that contains all records that you wish to insert



Parameterized Queries & Security

- **SQL Injection Prevention:**
 - Always use parameterized queries with `%s` placeholders instead of string formatting
- this prevents malicious SQL code injection attacks
- **Automatic Data Escaping:**
 - The MySQL connector automatically escapes special characters in parameters, ensuring data integrity and security without manual string manipulation
- **Performance Benefits:**
 - Parameterized queries can be cached and reused by the database engine, improving execution speed for repeated operations
- **Correct Syntax:**
 - Use `cursor.execute(query, (param1, param2))` with tuple parameters, never `cursor.execute(f"SELECT * FROM table WHERE id = {user_input}")`



What is SQL Injection?

- **Definition:** Malicious SQL code inserted through user input that gets executed by the database
- **Root cause:** Concatenating user input directly into SQL queries without proper sanitization
- **Attack vector:** Attackers exploit input fields to manipulate database queries beyond intended functionality
- **Impact severity:** Can lead to data theft, data corruption, unauthorized access, or complete database compromise
- **Prevention:** Always use parameterized queries with placeholder values (%s) instead of string concatenation



Vulnerable Code Example

- **Dangerous approach:** `query = f"SELECT * FROM users WHERE username = '{user_input}'"`
- **Attack scenario:** User enters `admin'; DROP TABLE users; --` as username
- **Resulting query:** `SELECT * FROM users WHERE username = 'admin'; DROP TABLE users; --'`
- **Devastating outcome:** The entire users table gets deleted from the database
- **Why it works:** Database treats the malicious input as legitimate SQL commands



Real Attack Example

- **Authentication bypass:** Input ' `OR '1'='1` makes any login succeed regardless of password
- **Data extraction:** Input ' `UNION SELECT password FROM users --` reveals all user passwords
- **Data modification:** Input ' `; UPDATE products SET price = 0 WHERE id = 1 --` changes product prices
- **System commands:** Advanced attacks can execute operating system commands through database functions
- **Mass data theft:** Attackers can dump entire databases containing sensitive customer information



Safe Parameterized Query Approach

- **Secure syntax:** `cursor.execute("SELECT * FROM users WHERE username = %s", (user_input,))`
- **Automatic escaping:** MySQL connector handles special characters and prevents code injection
- **Input treatment:** User input treated as data values, never as executable SQL code
- **Performance benefit:** Database can cache and reuse parameterized query execution plans
- **Best practice:** Always use %s placeholders with tuple parameters for all user-provided values



Code Organization Best Practices

- **Separation of concerns:** Database logic separate from business logic and user interface
- **Reusability:** Common database operations can be shared across multiple scripts
- **Maintainability:** Centralized connection management makes updates easier
- **Testing advantages:** Isolated database functions are easier to unit test and debug
- **Team collaboration:** Standardized modules ensure consistent database interaction patterns



What is a Module?

- A **self-contained, reusable unit of code** that encapsulates related functionality and can be imported across different parts of applications
- Think of it as a **specialized toolkit** that provides specific services (e.g., database connections, query execution, data validation)



Benefits of a Module

- **Code Reusability:** Write once, use everywhere - eliminate duplicate code across projects
- **Centralized Management:** All configurations, settings, and related functionality in one organized location
- **Enhanced Reliability:** Built-in error handling, resource cleanup, and robust exception management
- **Improved Testing & Maintenance:** Easier to test, debug, and update isolated components independently



Create a Module for Establishing DB Connection

```
db_connection.py U x
python > db_connection.py > ...
1  import os
2  from dotenv import load_dotenv
3  from mysql.connector import connect
4
5  # Load environment variables from .env file
6  env_path = os.path.join(os.path.dirname(__file__), ".env")
7  load_dotenv(dotenv_path=env_path, override=True)
8
9
10 def get_db_connection():
11     """Create database connection with proper configuration"""
12     return connect(
13         host=os.environ.get("DB_HOST", "localhost"),
14         user=os.environ["DB_USERNAME"],
15         password=os.environ["DB_PASSWORD"],
16         database=os.environ.get("DB_NAME", "movies_db"),
17         autocommit=False,
18         charset="utf8mb4",
19     )
20
```



Inserting Movies Using Method .executemany() (1/2)

```
insert_movies.py U x
python > insert_movies.py > insert_movies
1 from mysql.connector import Error
2 from python.db_connection import get_db_connection
3
4
5 def insert_movies():
6     """Insert movie data with proper connection handling"""
7     connection = None
8     cursor = None
9
10    try:
11        connection = get_db_connection()
12        cursor = connection.cursor()
13
14        print(f"Connected successfully to {connection.server_host}")
15
16        # Use parameterized query for security
17        insert_movies_query = """
18            INSERT INTO movies (title, release_year, genre, collection_in_mil)
19            VALUES (%s, %s, %s, %s)
20        """
21
22        movies_data = [
23            ("Forrest Gump", 1994, "Drama", 330.2),
24            ("3 Idiots", 2009, "Drama", 2.4),
25        ]
```

```
insert_movies.py U x
python > insert_movies.py > ...
27         print("Inserting data into table movies...")
28         cursor.executemany(insert_movies_query, movies_data)
29         connection.commit()
30         print(f"Successfully inserted {cursor.rowcount} movies into table 'movies'")
31
32     except Error as e:
33         print(f"Database Error: {e}")
34         if connection:
35             connection.rollback()
36         raise
37     except Exception as e:
38         print(f"Unexpected Error: {e}")
39         if connection:
40             connection.rollback()
41         raise
42     finally:
43         if cursor:
44             cursor.close()
45         if connection and connection.is_connected():
46             connection.close()
47         print("Database connection closed")
48
49
50 if __name__ == "__main__":
51     insert_movies()
```



Inserting Movies Using Method .executemany() (2/2)

```
● ~/courses/en813304/exercises/kanda-en813304-db-exerci
env_path is /Users/kandasaikaew/courses/en813304/exer
Connected successfully to localhost
Inserting data into table movies...
Successfully inserted 2 movies into table 'movies'
Database connection closed
```

```
○ ~/courses/en813304/exercises/kanda-en813304-db-exercises/python$mysql -u root
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 35
Server version: 9.3.0 Homebrew

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> use movies_db;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> select * from movies;
+-----+-----+-----+-----+-----+
| id | title       | release_year | genre | collection_in_mil |
+-----+-----+-----+-----+-----+
| 5  | Forrest Gump | 1994         | Drama | 330               |
| 6  | 3 Idiots    | 2009         | Drama | 2                 |
+-----+-----+-----+-----+-----+
2 rows in set (0.000 sec)
```



Inserting Reviewers

```
insert_reviewers.py U X
python > insert_reviewers.py > insert_reviewers

5 def insert_reviewers():
6     """Insert reviewer data with proper connection handling"""
7     connection = None
8     cursor = None
9
10    try:
11        connection = get_db_connection()
12        cursor = connection.cursor()
13
14        print(f"Connected successfully to {connection.server_host}")
15
16        # Use parameterized query for security
17        insert_reviewers_query = """
18            INSERT INTO reviewers (first_name, last_name)
19            VALUES (%s, %s)
20        """
21
22        reviewers_data = [
23            ("Chaitanya", "Baweja"),
24            ("Mary", "Cooper"),
25            ("John", "Wayne"),
26            ("Thomas", "Stoneman"),
27            ("Penny", "Hofstadter"),
28        ]
29
30        print("Inserting data into table reviewers...")
31        cursor.executemany(insert_reviewers_query, reviewers_data)
32        connection.commit()
33        print(
34            f"Successfully inserted {cursor.rowcount} reviewers into table 'reviewers'"
35        )
36
37    except Error as e:
38        print(f"Database Error: {e}")
39        if connection:
40            connection.rollback()
41        raise
```

```
~/courses/en813304/exercises/kanda-en813304-db-exercises/python$python3 insert_reviewers.py
Connected successfully to localhost
Inserting data into table reviewers...
Successfully inserted 5 reviewers into table 'reviewers'
Database connection closed
~/courses/en813304/exercises/kanda-en813304-db-exercises/python$mysql -u root
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 56
Server version: 9.3.0 Homebrew

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> use movies_db;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> select * from reviewers;
+----+-----+-----+
| id | first_name | last_name |
+----+-----+-----+
| 1  | Chaitanya  | Baweja    |
| 2  | Mary       | Cooper     |
| 3  | John       | Wayne     |
| 4  | Thomas     | Stoneman  |
| 5  | Penny      | Hofstadter |
```



Inserting Ratings

```
python > insert_ratings.py X
python > insert_ratings.py > ...
4
5 def insert_ratings():
6     """Insert rating data with proper connection handling"""
7     connection = None
8     cursor = None
9
10    try:
11        connection = get_db_connection()
12        cursor = connection.cursor()
13
14        print(f"Connected successfully to {connection.server_host}")
15
16        # Use parameterized query for security
17        insert_ratings_query = """
18            INSERT INTO ratings (rating, movie_id, reviewer_id)
19            VALUES (%s, %s, %s)
20        """
21
22        ratings_data = [
23            (6.4, 1, 1),
24            (5.6, 2, 2),
25            (6.3, 1, 3),
26            (5.1, 2, 4),
27            (5.0, 1, 5),
28        ]
29
30        print("Inserting data into table ratings...")
31        cursor.executemany(insert_ratings_query, ratings_data)
32        connection.commit()
33        print(f"Successfully inserted {cursor.rowcount} ratings into table 'ratings'")
34
35    except Error as e:
36        print(f"Database Error: {e}")
37        if connection:
38            connection.rollback()
39        raise
```

```
~/courses/en813304/exercises/kanda-en813304-db-exercises/python$python3 insert_ratings.py
Connected successfully to localhost
Inserting data into table ratings...
Successfully inserted 5 ratings into table 'ratings'
Database connection closed
~/courses/en813304/exercises/kanda-en813304-db-exercises/python$mysql -u root
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 58
Server version: 9.3.0 Homebrew
```

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its affiliates. Other names may be trademarks of their respective owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

```
[mysql> use movies_db;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A
```

Database changed

```
[mysql> select * from ratings;
```

movie_id	reviewer_id	rating
1	1	6.4
1	3	6.3
1	5	5.0
2	2	5.6
2	4	5.1



Reading Records Using SELECT Statement (1/3)

- To retrieve records, you need to send a SELECT query to `cursor.execute()`
- Then you use `cursor.fetchall()` to extract the retrieved table in the form of a list of rows or records.

```
read_data.py M X
python > read_data.py > select_movies
You, 1 minute ago | 1 author (You)
1 import os
2 from dotenv import load_dotenv
3 from mysql.connector import connect, Error
4
5 # Load environment variables from .env file
6 env_path = os.path.join(os.path.dirname(__file__), ".env")
7 load_dotenv(dotenv_path=env_path, override=True)
8
9
10 def get_db_connection():
11     """Create database connection with proper configuration"""
12     return connect(
13         host=os.environ.get("DB_HOST", "localhost"),
14         user=os.environ["DB_USERNAME"],
15         password=os.environ["DB_PASSWORD"],
16         database=os.environ.get("DB_NAME", "movies_db"),
17         autocommit=False,
18         charset="utf8mb4",
19     )
20
```



Reading Records Using SELECT Statement (2/3)

```
read_data.py M x
python > read_data.py > select_movies
22 def select_movies():
23     """Select and display all movies from the database"""
24     try:
25         with get_db_connection() as connection:
26             with connection.cursor() as cursor:
27                 select_movies_query = "SELECT * FROM movies ORDER BY release_year DESC"
28                 print("Executing query to fetch all movies...")
29                 cursor.execute(select_movies_query)
30                 result = cursor.fetchall()
31
32             if result:
33                 print(f"\nFound {len(result)} movies:")
34                 print("-" * 80)
35
36                 # Get column names for better display
37                 column_names = [desc[0] for desc in cursor.description]
38                 print(
39                     f"{'ID':<5} {'Title':<25} {'Year':<6} {'Genre':<15} {'Collection (M)'"
40                     f":<15}"
41                 )
41                 print("-" * 80)
```



Reading Records Using SELECT Statement (2/3)

```
read_data.py M x
python > read_data.py > select_movies

42         for row in result:
43             print(
44                 f"{row[0]:<5} {row[1]:<25} {row[2]:<6} {row[3]:<15} ${row[4]:<14}"
45             )
46         else:
47             print("No movies found in the database")
48             print(f"\nReading data from table 'movies' completed successfully")
49     except Error as e:
50         print(f"Database Error: {e}")
51         if hasattr(e, "errno"):
52             print(f"Error Code: {e.errno}")
53     except Exception as e:
54         print(f"Unexpected Error: {e}")
55     finally:
56         print("Database operation completed")
57
58
59 if __name__ == "__main__":
60     select_movies()
61
```

~/courses/en813304/exercises/kanda-en813304-db-exercises/python\$python3 read_data.py
Executing query to fetch all movies...

Found 2 movies:

ID	Title	Year	Genre	Collection (M)
6	3 Idiots	2009	Drama	\$2
5	Forrest Gump	1994	Drama	\$330

Reading data from table 'movies' completed successfully
Database operation completed



Filtering Results Using the WHERE Clause

- You can filter table records by specific criteria using the WHERE clause
- For example, to retrieve all movies with a box office collection greater than \$300 million, you could run the following query:

```
filter_data.py M X
python > filter_data.py > ...
You, 1 second ago | 1 author (You)
1 from mysql.connector import Error
2 from python.db_connection import get_db_connection
3
4 | You, 1 second ago • Uncommitted changes
5 def get_high_grossing_movies(min_collection=300):
6     """Get movies with collection above threshold."""
7
8     query = """
9     SELECT title, collection_in_mil
10    FROM movies
11    WHERE collection_in_mil > %s
12    ORDER BY collection_in_mil DESC
13    """
14
15    try:
16        with get_db_connection() as connection:
17            with connection.cursor() as cursor:
18                cursor.execute(query, (min_collection,))
19                return cursor.fetchall()
20    except Error as e:
21        print(f"Database Error: {e}")
```

```
~/courses/en813304/exercises/kanda-en813304-db-exercises/python$python3 filter_data.py
```

Forrest Gump: \$330M

Found 1 high-grossing movies



Using .fetchone() and .fetchmany()

- `.fetchone()` retrieves either the next row of the result, as a tuple, or `None` if no more rows are available.
- `.fetchmany()` retrieves the next set of rows from the result as a list of tuples.
 - It has a size argument, which defaults to 1, that you can use to specify the number of rows you need to fetch.
 - If no more rows are available, then the method returns an empty list.



Example of Using .fetchmany() (1/2)

fetch_many.py M X

python > fetch_many.py > get_movies

You, 33 minutes ago | 1 author (You)

```
1 from mysql.connector import Error
2 from db_connection import get_db_connection
3
4
5 def get_movies(limit=2):
6     """Get movies from the database with optional limit."""
7
8     query = """
9     SELECT *
10    FROM movies
11    LIMIT %s
12    """
13
14    try:
15        with get_db_connection() as connection:
16            with connection.cursor() as cursor:
17                cursor.execute(query, (limit,))
18                return cursor.fetchall()
19    except Error as e:
20        print(f"Database Error: {e}")
21    return []
```

fetch_many.py M X

python > fetch_many.py > get_movies

```
24 def get_all_movies():
25     """Get all movies from the database."""
26
27     query = """
28     SELECT *
29     FROM movies
30     """
31
32     try:
33         with get_db_connection() as connection:
34             with connection.cursor() as cursor:
35                 cursor.execute(query)
36                 return cursor.fetchall()
37     except Error as e:
38         print(f"Database Error: {e}")
39     return []
40
41
```



Example of Using .fetchmany() (2/2)

fetch_many.py M X

▶ ◂ ◃

python > fetch_many.py > get_movies

```
42 def main():
43     try:
44         # Get limited number of movies
45         movies = get_movies(2)
46
47         print("Movies (limited):")
48         for row in movies:
49             print(row)
50
51         print(f"\nFound {len(movies)} movies")
52         print("Filtering data from table 'movies' successfully")
53
54     except ValueError as e:
55         print(f"Error: {e}")
56
57
58 if __name__ == "__main__":
59     main()
```

~/courses/en813304/exercises/kanda-en813304-db-exercises/python\$python3 fetch_many.py

Movies (limited):

(5, 'Forrest Gump', 1994, 'Drama', 330)

(6, '3 Idiots', 2009, 'Drama', 2)

Found 2 movies

Filtering data from table 'movies' successfully



Working with Related Data

- **Real-world complexity:** Most applications require data from multiple related tables
- **Relational design:** Our movie database demonstrates foreign key relationships between entities
- **JOIN operations:** Combine data from movies, reviewers, and ratings tables in single queries
- **Data integrity:** Foreign key constraints ensure ratings only reference existing movies and reviewers
- **Query efficiency:** JOINS are more efficient than separate queries with manual data combination
-



Handling Multiple Tables Using the JOIN Statement

- If you found the queries in the last section to be quite straightforward, don't worry.
- You can make your SELECT queries as complex as you want using the same methods from the last section.

```
~/courses/en813304/exercises/kanda-en813304-db-exercises/python$python3 join_data.py
Connected successfully to localhost
Executing query to get movie ratings...
Found 2 movies with ratings:
-----
Forrest Gump: 5.90
3 Idiots: 5.35
Query executed successfully
Database connection closed
```

```
join_data.py > ...
19 join_movies_query = """
20 SELECT title, AVG(rating) as
   average_rating
21 FROM ratings
22 INNER JOIN movies
23 ON movies.id = ratings.movie_id
24 GROUP BY movie_id
25 ORDER BY average_rating DESC
26 """
27
28 with connection.cursor() as cursor:
29     cursor.execute(join_movies_query)
30     result = cursor.fetchall()
31     for row in result:
32         print(row)
33     print("Joining data successfully")
34 except Error as e:
35     print(f"Database Error: {e}")
36 except ValueError as e:
37     print(f"Configuration Error: {e}")
38
```



UPDATE Statement

```
update_data.py M X
python > update_data.py > update_data

5 def update_data():      You, 1 second ago • Uncommitted changes
6     connection = None
7     cursor = None
8     try:
9         connection = get_db_connection()
10        cursor = connection.cursor()
11
12        print(f"Connected successfully to {connection.server_host}")
13        movie_id = input("Enter movie id: ")
14        reviewer_id = input("Enter reviewer id: ")
15        new_rating = input("Enter new rating: ")
16        update_query = """
17        UPDATE ratings
18        SET rating = "%s"
19        WHERE movie_id = "%s" AND reviewer_id = "%s";
20
21        SELECT *
22        FROM ratings
23        WHERE
24            movie_id = "%s" AND reviewer_id = "%s"
25        """ % (
26            new_rating,
27            movie_id,
28            reviewer_id,
29            movie_id,
30            reviewer_id,
31        )
```

```
update_data.py M X
python > update_data.py > update_data

32     with connection.cursor() as cursor:
33         for result in cursor.execute(update_query, multi=True):
34             if result.with_rows:
35                 print(result.fetchall())
36             connection.commit()
37     except Error as e:
38         print(f"Database Error: {e}")
39         if connection:
40             connection.rollback()
41         raise
42     except Exception as e:
43         print(f"Unexpected Error: {e}")
44         if connection:
45             connection.rollback()
46         raise
47     finally:
48         if cursor:
49             cursor.close()
50         if connection and connection.is_connected():
51             connection.close()
52         print("Database connection closed")
53
54
55 if __name__ == "__main__":
56     update_data()
57
```

To pass multiple queries to a single `cursor.execute()`, you need to set the method's **multi** argument to **True**.

```
~/courses/en813304/exercises/kanda-en813304-db-exercises/python$python3 update_data.py
Connected successfully to localhost
Enter movie id: 1
Enter reviewer id: 1
Enter new rating: 8
[(1, 1, Decimal('8.0'))]
Database connection closed
```



DELETE Statement

```
db_connection.py U  crt_tables.py  describe_tables.py U  delete_data.py M X
python > delete_data.py > delete_data

1 from mysql.connector import Error
2 from db_connection import get_db_connection
3
4
5 def delete_data():
6     connection = None
7     cursor = None
8     try:
9         connection = get_db_connection()
10
11         print(f"Connected successfully to {connection.server_host}")
12         delete_query = """
13         DELETE FROM ratings WHERE reviewer_id = 2
14         """
15         with connection.cursor() as cursor:
16             cursor.execute(delete_query)
17             connection.commit()
18     except Error as e:
19         print(f"Database Error: {e}")
20         if connection:
21             connection.rollback()
22         raise
23     except Exception as e:
24         print(f"Unexpected Error: {e}")
25         if connection:
26             connection.rollback()
27         raise
28 finally:
29     if cursor:
30         cursor.close()
31     if connection and connection.is_connected():
32         connection.close()
33     print("Database connection closed")
```

```
[mysql> select * from ratings;
```

movie_id	reviewer_id	rating
1	1	6.4
1	3	6.3
1	5	5.0
2	2	5.6
2	4	5.1

```
5 rows in set (0.001 sec)
```

```
[mysql> select * from ratings;
```

movie_id	reviewer_id	rating
1	1	6.4
1	3	6.3
1	5	5.0
2	4	5.1

```
4 rows in set (0.001 sec)
```



Conclusions

- MySQL Connector/Python enables Python applications to interact with MySQL databases
- Basic database operations (CREATE, INSERT, SELECT, UPDATE, DELETE) can be performed using Python code
- Use `.execute()` for single queries and `.executemany()` for efficiently inserting multiple records
- Proper connection handling and error management are crucial for robust database applications
- Complex queries involving multiple tables can be executed using JOIN statements and filtered results



References

- <https://realpython.com/python-mysql/>
- <https://claude.ai>

